

Lesson 11 — Homework

Modules və Packages

Tapşırıq: Book Management System

Sadə **Book Management System** hazırlayın.

Bu dəfə layihənin folder strukturunu, faylların adlarını və funksiyaların hansı module-da yerləşəcəyini özünüz müəyyən etməlisiniz.

Məqsəd yalnız proqramın işləməsi deyil. Kodun **modules və packages istifadə edilərək düzgün şəkildə təşkil olunması** da qiymətləndiriləcək.

Dərstdən qısa xatırlatma

Aşağıdakı hissə homework-un həlli deyil. Tapşırığı hazırlayarkən dərstdə keçdiyimiz mövzuları xatırlamaq üçün istifadə edə bilərsiniz.

Module nədir?

Python-da hər .py faylı bir **module** ola bilər.

Məsələn:

```
main.py
users.py
validators.py
```

Burada hər Python faylı ayrıca module-dur.

Module-ların əsas məqsədlərindən biri böyük kodu daha kiçik və məntiqli hissələrə bölməkdir.

Məsələn, bir layihədə bütün funksiyaları `main.py` daxilində saxlamaq əvəzinə, əlaqəli funksiyalar ayrı module-larda yerləşdirilə bilər.

Başqa module-dan necə istifadə edilir?

Bir module-u başqa Python faylında istifadə etmək üçün `import` edirik.

Bütün module-u import etmək

```
import users
```

Daha sonra:

```
users.show_users()
```

Burada funksiyanın hansı module-dan gəldiyi açıq şəkildə görünür.

Module-dan müəyyən bir şeyi import etmək

```
from users import show_users
```

Daha sonra funksiyanı birbaşa istifadə etmək mümkündür:

```
show_users()
```

Bir neçə funksiya da import etmək olar:

```
from users import add_user, show_users
```

as nə üçün istifadə olunur?

Import edilən module və ya funksiya başqa ad vermək mümkündür.

```
import users as user_service
```

Sonra:

```
user_service.show_users()
```

və ya:

```
from users import add_user as create_user
```

Bu üsul uzun adları qısaltmaq və ya ad qarışıqlığının qarşısını almaq üçün istifadə edilə bilər.

Package nədir?

Package əlaqəli module-ları bir folder daxilində qruplaşdırmağa imkan verir.

Məsələn:

```
project/
├── main.py
└── app/
    ├── __init__.py
    ├── module1.py
    └── module2.py
```

Burada:

```
app
```

bir package-dir.

Package-in daxilində bir neçə module yerləşə bilər.

Bu, böyük layihələrin strukturunu daha səliqəli saxlamağa kömək edir.

Homework-da package-in və module-ların adlarını özünüz müəyyən etməlisiniz.

__init__.py nədir?

Python package-lərində tez-tez:

```
__init__.py
```

faylı görəcəksiniz.

Sadə layihələrdə bu fayl boş qala bilər.

Məsələn:

```
app/
├── __init__.py
├── users.py
└── validators.py
```

Bu dərsdə __init__.py daxilində əlavə kod yazmağınız tələb olunmur.

Package daxilindən import

Əgər bir module package daxilindədirsə, onu package adı ilə import etmək mümkündür.

Ümumi nümunə:

```
from package_name.module_name import function_name
```

Məsələn:

```
from app.users import add_user
```

Burada:

- app → package
- users → module
- add_user → həmin module daxilində olan funksiya

Kod niyə ayrı module-lara bölünür?

Module yaratmağın məqsədi sadəcə çoxlu .py faylı yaratmaq deyil.

Hər module-un müəyyən bir **məsuliyyəti** olmalıdır.

Məsələn, nəzəri olaraq bir layihədə:

```
validation
statistics
data operations
main program
```

kimi fərqli məsuliyyətlər ola bilər.

Amma hansı hissənin hansı module-da saxlanacağına layihəyə uyğun qərar vermək lazımdır.

Homework-da bu qərarı siz verməlisiniz.

Əgər funksiyalar tamamilə fərqli iş görürsə, onların ayrı module-larda olması daha məntiqli ola bilər.

return və print fərqlərini unutmayın

Funksiya daxilində:

```
print(result)
```

nəticəni sadəcə ekrana göstərir.

Amma:

```
return result
```

nəticəni funksiyanı çağıran hissəyə qaytarır.

Məsələn:

```
result = some_function()
```

yazmaq istəyirsinizsə, some_function() çox vaxt nəticəni return etməlidir.

Axtarış və hesablama funksiyalarını yazarkən bu fərqi diqqət edin.

if __name__ == "__main__": necə işləyir?

Python hər module üçün avtomatik olaraq:

```
__name__
```

adlı xüsusi variable yaradır.

Əgər fayl birbaşa run edilsə:

```
__name__ == "__main__"
```

olur.

Buna görə proqramın əsas işləmə hissəsini belə yazı bilərik:

```
if __name__ == "__main__":  
    # proqramı başladan kod  
    pass
```

Bu hissə həmin fayl başqa module tərəfindən import edildikdə avtomatik işləməyəcək.

Bu yanaşma funksiyaları proqramı başladan koddan ayırmağa kömək edir.

Layihəni başlamazdan əvvəl

Birbaşa kod yazmağa başlamazdan əvvəl 2-3 dəqiqə düşünüb layihəni planlaşdırın.

Özünüə aşağıdakı sualları verin:

1. Proqramda hansı əməliyyatlar olacaq?
2. Hansı funksiyalar bir-biri ilə əlaqəlidir?
3. Hansı funksiyalar ayrı məsuliyyət daşıyır?
4. Neçə module yaratmaq məntiqlidir?
5. Package daxilində hansı module-lar yerləşməlidir?
6. Proqram hansı fayldan başladılmalıdır?
7. Hansı module hansı module-dan import edilməlidir?

Əvvəl strukturu düşünmək, sonra kod yazmaq böyük layihələrdə işinizi xeyli asanlaşdıracaq.

Sistem tələbləri

Sistem kitabların idarə olunmasına imkan verməlidir.

Hər kitab haqqında aşağıdakı məlumatlar saxlanmalıdır:

- kitabın adı;
- müəllif;
- buraxılış ili.

Kitab məlumatlarını saxlamaq üçün dictionary istifadə edə bilərsiniz.

Bütün kitablar bir list daxilində saxlanmalıdır.

1. Kitab əlavə etmək

Sistemdə yeni kitab əlavə etmək mümkün olmalıdır.

Məsələn:

```
Title: 1984
Author: George Orwell
Year: 1949
```

Kitab əlavə edilməzdən əvvəl daxil edilən məlumatlar yoxlanmalıdır.

Validation qaydaları

Kitabın adı:

- boş ola bilməz;
- minimum 2 simvol olmalıdır.

Müəllif:

- boş ola bilməz;
- minimum 2 simvol olmalıdır.

Buraxılış ili:

- 0-dan böyük olmalıdır;
- 2026-dan böyük ola bilməz.

Əgər məlumatlardan hər hansı biri düzgün deyilsə, kitab sistemə əlavə edilməməlidir.

İstifadəçiyə uyğun mesaj göstərilməlidir.

Məsələn:

```
Kitab məlumatları düzgün deyil.
```

2. Bütün kitabları göstərmək

Sistemdə olan bütün kitabları göstərmək mümkün olmalıdır.

Nəticə məsələn belə görünə bilər:

```
1984 | George Orwell | 1949
The Hobbit | J.R.R. Tolkien | 1937
Clean Code | Robert C. Martin | 2008
```

Output formatını özünüz müəyyən edə bilərsiniz.

Əsas tələb məlumatların oxunaqlı olmasıdır.

3. Kitabı adına görə axtarmaq

İstifadəçi kitabın adını verərək həmin kitabı tapa bilməlidir.

Məsələn:

1984

Kitab tapılırsa onun məlumatları göstərilməlidir.

Kitab tapılmazsa:

Kitab tapılmadı.

kimi uyğun mesaj göstərilməlidir.

4. Müəllifə görə kitabları tapmaq

Sistem verilmiş müəllifin bütün kitablarını tapa bilməlidir.

Məsələn sistemdə:

1984 | George Orwell | 1949
Animal Farm | George Orwell | 1945
The Hobbit | J.R.R. Tolkien | 1937

varsə və:

George Orwell

üzrə axtarış edildirsə, nəticədə həmin müəllifin hər iki kitabı qaytarılmalıdır.

Əgər həmin müəllifə aid heç bir kitab yoxdursa, uyğun nəticə qaytarılmalıdır.

5. Statistik məlumatlar

Sistem aşağıdakı statistikaları hesablaya bilməlidir:

Ümumi kitab sayı

Məsələn:

Ümumi kitab sayı: 5

Köhnə kitabların sayı

2000-ci ildən əvvəl buraxılmış kitabların sayı hesablanmalıdır.

Məsələn:

2000-ci ildən əvvəl çıxan kitabların sayı: 3

Kitabların orta buraxılış ili

Sistemdəki bütün kitabların buraxılış illərinin ortalaması hesablanmalıdır.

Məsələn:

Orta buraxılış ili: 1987.5

Sistemdə heç bir kitab yoxdursa, proqram error verməməlidir.

6. Test məlumatları

Proqramınızı yoxlamaq üçün ən azı **5 kitab** əlavə edin.

Onların arasında:

- ən azı 2 kitab 2000-ci ildən əvvəl;

- ən azı 2 kitab 2000-ci ildən sonra;
- eyni müəllifə aid ən azı 2 kitab olmalıdır.

7. Modules və Packages

Bu homework-un əsas mövzusu **Modules və Packages** olduğu üçün bütün proqramı bir Python faylında yazmaq olmaz.

Layihəni məntiqi hissələrə bölün.

Özünüz qərar verin:

- neçə module olacaq;
- module-ların adları nə olacaq;
- hansı funksiyalar hansı module-da yerləşəcək;
- hansı module-lar package daxilində olacaq;
- hansı import üsullarından istifadə edəcəksiniz;
- proqramın əsas işləmə nöqtəsi hansı fayl olacaq.

Kodun strukturu aydın və məntiqli olmalıdır.

8. Import

Module-lar arasında lazım olan funksiyaları düzgün şəkildə import edin.

Dərsdə öyrəndiyimiz üsullardan istifadə edə bilərsiniz:

```
import module_name
```

və ya:

```
from module_name import function_name
```

Package daxilində olan module-ları da düzgün şəkildə import etməlisiniz.

9. __name__ == "__main__"

Layihədə uyğun yerdə:

```
if __name__ == "__main__":
```

istifadə edin.

Məqsəd proqramın əsas işləmə hissəsini module daxilində olan funksiyalardan ayırmaqdır.

Final Task

Sistemə **kitab silmək** funksionallığı əlavə edin.

İstifadəçi kitabın adını verdikdə:

- kitab tapılırsa sistemdən silinməlidir;
- kitab tapılmazsa uyğun mesaj göstərilməlidir.

Məsələn:

```
Silinəcək kitab: 1984
```

Nəticə:

```
1984 kitabı silindi.
```

və ya:

Kitab tapılmadı.

Bu funksionallığın layihənin hansı hissəsində yerləşəcəyinə özünüz qərar verin.

Gözlənilən proqram axını

Proqram işə düşdükdə ən azı aşağıdakı əməliyyatlar nümayiş etdirilməlidir:

1. Bir neçə kitab əlavə edilir.
2. Bütün kitablar göstərilir.
3. Kitab adına görə axtarış edilir.
4. Müəllifə görə axtarış edilir.
5. Ümumi kitab sayı hesablanır.
6. 2000-ci ildən əvvəl çıxmış kitabların sayı hesablanır.
7. Orta buraxılış ili hesablanır.
8. Bir kitab silinir.
9. Silinmiş kitabın artıq list-də olmadığı göstərilir.

Əsas qaydalar

- Bütün proqramı bir .py faylında yazmaq olmaz.
- Modules istifadə edilməlidir.
- Ən azı bir package istifadə edilməlidir.
- Funksiyalardan istifadə edilməlidir.
- Module-lar arasında import olmalıdır.
- Validation ayrıca məntiq kimi düşünülməlidir.
- `if __name__ == "__main__":` istifadə edilməlidir.
- Proqram düzgün olmayan məlumat daxil edildikdə error verməməlidir.
- Kod təkrarı mümkün qədər az olmalıdır.
- Variable və funksiya adları aydın olmalıdır.